05
Image Captioning

Introduction

Image captioning is the process of generating text that describes an image. The generated text, also known as caption, should accurately reflect the image's content.

Image captioning has multiple applications. For example, on social media platforms, it automatically suggests image captions, saving time for content creators. In online retail, it generates captions for product images, thus improving the shopping experience.

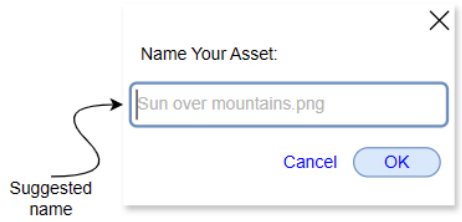


Figure 1: Image captioning system suggests file names for an uploaded image

Beyond user-facing applications, image captioning is also used in systems that operate behind the scenes. For instance, in NSFW (Not Safe for Work) content moderation, image captioning systems can generate descriptive captions that help identify and flag inappropriate or explicit content by providing text-based interpretations of images. Additionally, image captioning can address the cold-start problem in recommendation systems, which occurs when a system lacks sufficient data on new users or items to make accurate recommendations. By generating descriptive captions, the system gains textual information that helps categorize and recommend new items based on their content.

In this chapter, we design a machine learning (ML) system that generates descriptive captions for images.

Clarifying Requirements

Here is a typical interaction between a candidate and an interviewer:

Candidate: There are various types of images, including general everyday images and domain-specific images such as medical imagery or technical diagrams. Can I focus on general everyday images?
Interviewer: Sure.

Candidate: Are there any specific applications or use cases we are targeting with this system?
Interviewer: We are targeting name suggestions to designers when they upload their assets.

Candidate: Since the image captioner will be used for asset name suggestions, the captions should not be too long and detailed. Is this a fair assumption?
Interviewer: Makes sense. The captions should be short, but descriptive and clear.

Candidate: Should the system support multiple languages, or will it focus only on English?
Interviewer: Let's focus on English only.

Candidate: What is the estimated size and diversity of the dataset?
Interviewer: We have access to a large dataset with 400 million image–caption pairs focused on everyday images.

Candidate: Does the dataset consist solely of English captions?
Interviewer: The dataset is not preprocessed. There might be captions in different languages, and some captions might be noisy or inaccurate. Additionally, captions for some images may be missing.

Candidate: Is real-time captioning required?
Interviewer: The system should generate a caption quickly, though real-time speed is not necessary. A latency of 1–2 seconds is acceptable.

Candidate: How should the system handle images with ambiguous content or unclear focus?
Interviewer: In such cases, the system should skip suggesting a caption.

Candidate: I assume the system should avoid generating biased captions or captions with offensive words. Is that a fair assumption?
Interviewer: Great point. Yes, it is crucial to ensure our system remains fair and safe for users.

Candidate: What are the typical image dimensions? Very small images can be unclear, leading to incorrect captions.
Interviewer: Let's assume the system only suggests names for images with a minimum resolution of 256 x 256 pixels.

Frame the Problem as an ML Task

Specifying the system's input and output

The input to an image captioning system is an image. This image is processed by the model to generate a descriptive caption. The output, therefore, is a text that accurately describes the content of the image.

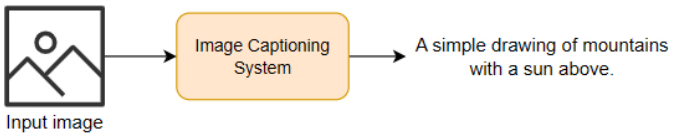


Figure 2: Input and output of an image captioning system

Choosing a suitable ML approach

The image captioning problem introduces a unique challenge: an ML model requires visual understanding to process the input image, language understanding to generate a caption, and the ability to bridge the gap between visual and textual modalities. This requires developing a multi-modal system.

A common approach to building multi-modal systems is to use an encoder-decoder framework. Similar to language translation—where we utilized an encoder-decoder architecture—we treat the image as a new "language" in this context. Specifically, we employ two main components, each handling one modality:

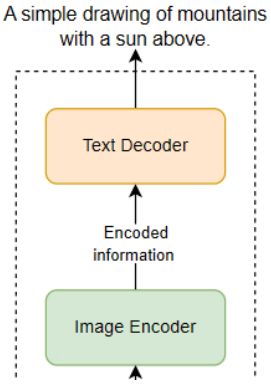
- Image encoder
- Text decoder

Image encoder

The image encoder is responsible for understanding the visual content of the image and encoding the image into a lower-dimensional representation.

Text decoder

The text decoder uses the encoded visual information from the image encoder to generate a descriptive caption.



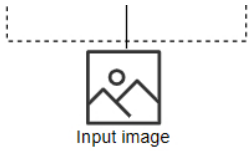


Figure 3: Image captioning components

We will explore the architecture of these components in detail in the model development section. It's important to note that there are various approaches to tackling the image captioning problem. While we focus on the encoder-decoder framework here, alternative models such as BLIP-2 [1], BLIP-3 [2], and InternVL [3] offer different techniques and architectures for generating captions. If you're interested in these other methods, you can refer to [1] [2] [3] for a broader understanding of the image captioning landscape.

Data Preparation

In this section, we prepare the dataset to train our image captioning system.

Image	Caption
	A simple drawing of mountains with a sun above.
	A minimalistic flower icon.
...	...

Figure 4: Example of image–caption dataset

The dataset comprises 400 million pairs of images and captions. However, not all images or captions are suitable for training. Let's examine data preparation for captions and images separately.

Caption preparation

Raw captions are often noisy and not in a format that is usable by the ML model. During caption preparation, we remove inappropriate captions and ensure the remaining ones are consistent and tokenized. In particular, we perform the following steps:

- **Remove pairs with a non-English caption:** We remove image–caption pairs where the caption is not in English, as this model's focus will be on English.
- **Remove duplicate images or captions:** To ensure the diversity and quality of the training data, we eliminate duplicate images and captions. Duplicate images are identified using perceptual hashing techniques or image similarity models (e.g., CLIP image encoder), while duplicate captions are detected by exact match or semantic similarity checks (e.g., CLIP text encoder). Removing duplicates prevents the model from overfitting to redundant data and helps it learn a broader range of associations between images and text.
- **Remove irrelevant captions:** We use a pretrained vision–language model (e.g., CLIP) to assess the relevance between images and their corresponding captions. A higher score usually indicates greater semantic relevance between the image and the text. We remove pairs with scores below a specific threshold, such as 0.25. This ensures our model learns from high-quality, relevant pairs. For more information on how CLIP scores the relevance between text and images, refer to Chapter 9.
- **Summarize long captions:** Captions are often long and detailed. Training the model with these captions leads to the generation of similarly long captions, which doesn't suit our use case. To address this, we summarize the captions using a large language model such as Llama [4] to create brief, concise descriptions that meet our requirements.
- **Normalize captions:** We apply standard text normalization techniques including lowercasing and trimming whitespaces to maintain consistency between captions.
- **Tokenize captions:** We use a subword-level tokenization algorithm such as Byte-Pair Encoding (BPE) [5] to tokenize captions into a sequence of IDs. For a detailed review of text tokenization methods and the BPE algorithm, refer to Chapter 2 and Chapter 3.

Image preparation

As is the case for captions, not all images are useful. We remove images that might hurt training and ensure the remaining images are consistent and suitable for the model training. In particular, we perform the following steps:

- **Remove low-resolution images:** We remove image–caption pairs in which the image resolution is less than 256× 256 because such low-resolution images might not provide enough detail for accurate caption generation.
- **Normalize images:** We scale the pixel values to a normalized range, such as 0 to 1. This normalization makes the training process more stable.
- **Remove low-quality images:** To maintain high-quality training data, we filter images that exhibit conditions such as blurriness, overexposure, underexposure, or other defects that degrade visual clarity. Image quality assessment methods, such as the LAION Aesthetics Predictor [6], help identify and remove subpar images by scoring them on factors such as sharpness, contrast, and lighting.
- **Adjust image dimensions:** Images typically have a range of sizes and aspect ratios. We resize all images to a uniform size. This is critical since ML models require fixed-size inputs during training. When adjusting image dimensions to a uniform size, it is important to preserve their original aspect ratios. To do so, we often follow two steps:

1. **Resizing:** First, we resize the image so that the smaller dimension matches the target size. For instance, if our target size is 256×256 and our original image is 512×768, we resize it to 256×384.
2. **Center-cropping:** Next, we center-crop the resized image to the target dimensions. From our previous example, we center-crop the 256×384 image to 256×256.

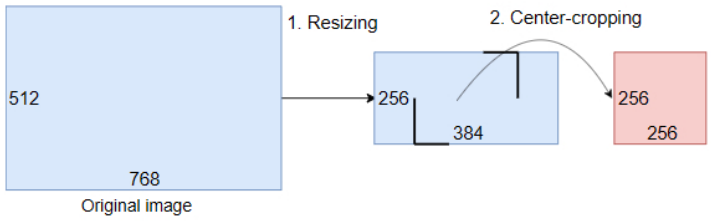


Figure 5: Resizing followed by center-cropping

This two-step method ensures our images maintain their aspect ratios and fit the required size for our ML model.

Model Development

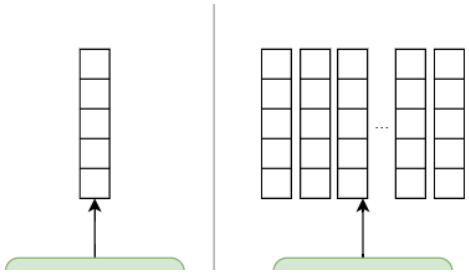
Architecture

We framed image captioning as a multi-modal language generation task where the image encoder processes an input image, and the text decoder generates a descriptive caption. In this section, we explore the architecture of the image encoder and text decoder.

Image encoder

The image encoder is responsible for processing an image and encoding the information within it.

The output of the encoder plays a pivotal role in determining the quality and specificity of the generated captions. The encoder's output can be either a single token, representing the entire image as a single feature vector, or a sequence of tokens, where each token corresponds to a specific region or aspect of the image. The choice between these two approaches has significant implications for how effectively the system captures and represents the visual content, and research has explored both options to understand their strengths and limitations.



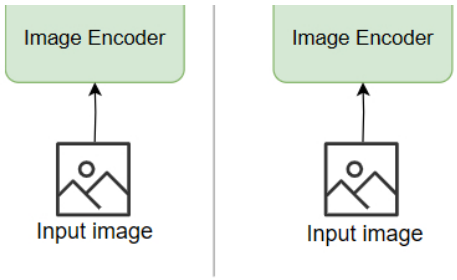


Figure 6: Image encoder outputting single token vs. sequence of tokens

When the encoder produces a single token as its output, it effectively compresses the entire image into one vector. This vector serves as a summary of the image, encapsulating its global features and overall context. The primary advantage of this approach lies in its simplicity; the architecture remains straightforward, with reduced computational complexity and lower resource requirements. A single vector emphasizes the overall content of the image, which can be particularly beneficial for generating concise and high-level captions that capture the general essence of the scene. However, this approach also has notable downsides. Condensing all visual information into one vector often means the loss of local details and specific nuances, which are crucial for generating descriptive and contextually rich captions. As a result, captions generated from single-token outputs may lean toward being more generic and may struggle with complex images that require detailed representation.

On the other hand, producing a sequence of tokens from the encoder allows the system to capture a more granular view of the image. Each token in the sequence corresponds to a distinct part or patch of the image, resulting in a richer and more comprehensive representation that includes both global and local features. This approach aligns particularly well with the attention mechanism, which is a cornerstone of modern generative models such as Transformers. The attention mechanism works best with sequence inputs, as it enables the decoder to focus dynamically on different regions of the image during caption generation. This capability of selectively attending to various parts of the image leads to more accurate, relevant, and detailed captions. By using a sequence of tokens, the model can generate captions that are not only more descriptive but also better aligned with the specific objects, actions, and contexts present in the image.

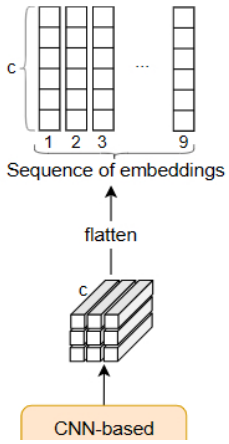
The image encoder architectures can be divided into the following:

- CNN-based
- Transformer-based

CNN-based

Convolutional Neural Networks (CNNs) are traditionally used for image-encoding tasks. CNNs excel at capturing spatial hierarchies in images through the use of convolutional filters. These filters detect patterns such as edges, textures, and objects at different scales.

CNN-based encoders process the input image and output a grid of feature vectors. For example, as shown in Figure 7, an input image passes through the CNN, producing a feature vector of size 3 x 3 x c. Here, c represents the channel size, which depends on the CNN architecture. While the CNN produces a 3 x 3 x c output, the Transformer in the text decoder needs a sequence of features (i.e., 9 x c). To achieve this, we use a flattening or reshaping operation that reorganizes the features from each of the nine positions in the 3 x 3 grid into a sequential format.



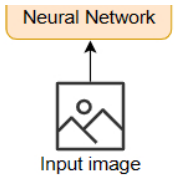


Figure 7: A CNN-based image encoding

Transformer-based

Transformer models, originally developed for natural language processing, have recently been adapted for image encoding with significant success. In this architecture, a Transformer analyzes images, extracts features, and encodes them into a sequence of embeddings. Specifically, a Transformer-based image encoder consists of:

- Patchify
- Positional encoding
- Transformer

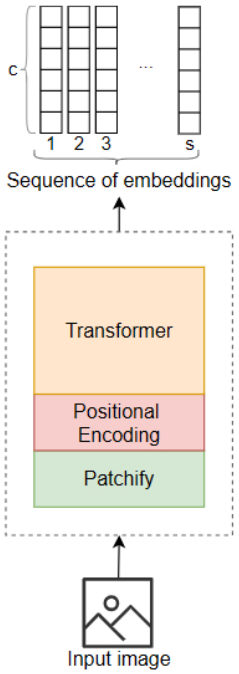


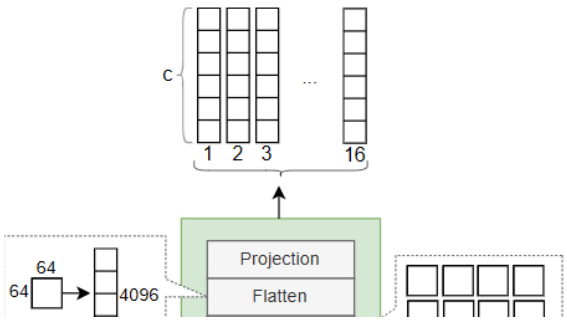
Figure 8: Transformer-based image encoding

Patchify

Since Transformers work with sequences, the image should first be converted into a sequence. This process involves three steps:

1. Divide the image into fixed-size patches
2. Flatten each patch
3. Linearly project each patch

For example, a 256 x 256 input image is divided into patches of 64 x 64. These patches are flattened into 4096-sized vectors and linearly projected into embedding vectors of size c, where c is the desired embedding size.



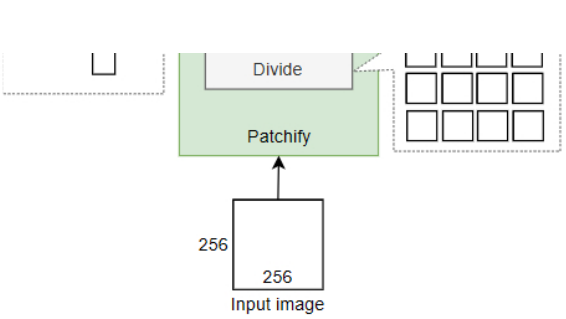


Figure 9: Patchification process

Positional encoding

Positional encoding assigns position information to each patch, specifying where each patch was located in the original image. This helps Transformers understand positions within the sequence.

Positional encoding can be implemented in various ways. Let's briefly explore the following variations:

- 1D vs. 2D positional encoding
- Learnable vs. fixed positional encoding

1D vs. 2D positional encoding

1D positional encoding employs a function that maps an integer (position in the sequence) to a c-dimensional vector, where c is usually the Transformer's hidden dimension. This is commonly used in text sequences, with each token receiving a positional vector based on its place. When applied to images, 1D positional encoding encodes the position of each patch in a flattened sequence, which might not capture the two-dimensional spatial relationships in images.

2D positional encoding, on the other hand, maps two integers—representing the row and column positions in the image grid—into a c-dimensional vector. This encoding method is more suitable for images as it preserves the spatial structure.

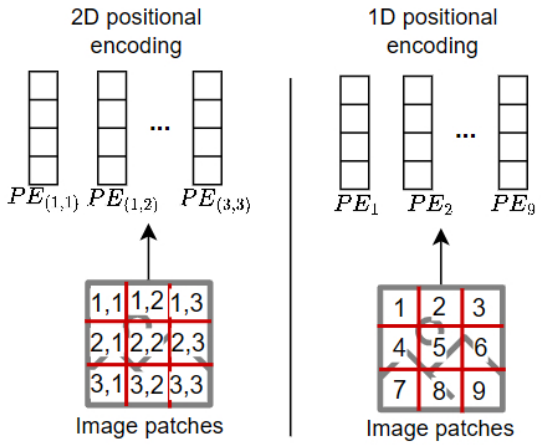


Figure 10: 1D vs. 2D positional encoding

Learnable vs. fixed positional encoding

In learnable positional encoding, the model learns positional encodings during training. A neural network maps positions (1D or 2D) to a c-dimensional vector. In the fixed approach, positional encodings are determined by a fixed function such as sine–cosine. For more details, refer to Chapter 2.

There is often no best solution when choosing between 1D vs. 2D and learnable vs. fixed positional encoding. While Vision Transformer (ViT) [7] uses learnable 1D positional encoding, in practice, we often test different combinations to see which works best for a specific task.

Which architecture is suitable for our image encoder?

CNNs are effective at capturing local patterns in images but they struggle with long-range dependencies between distant regions of the image. In contrast, Transformers capture both local and global relationships in the image using a

self-attention mechanism. This allows Transformers to model complex dependencies, making them ideal for tasks that require detailed, context-aware image understanding, for example, generating descriptive captions. For these reasons, we follow the ViT [7] and choose Transformer-based architecture as our image encoder.

Text decoder

The text decoder is responsible for generating the caption. As we saw in previous chapters, a decoder-only Transformer is the standard choice for text generation. The input to the decoder-only Transformer is a sequence of vectors corresponding to the input image. Its output is the caption, generated one token at a time.

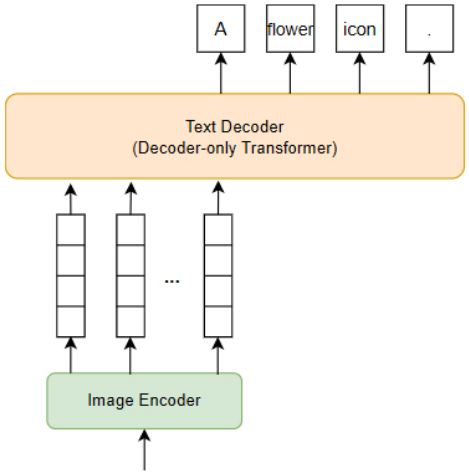


Figure 11: Providing image as a sequence of embeddings

Training

The training approach for the image captioning model is similar to the strategies discussed in previous chapters. We follow a two-stage training strategy:

1. Unsupervised pretraining
2. Supervised finetuning

1. Unsupervised pretraining

During this stage, the text decoder – which is a decoder-only Transformer – is trained on general data. The purpose of this stage is to develop a base model that has a broad understanding of language structure and is capable of generating coherent text. This knowledge is crucial for the model to perform well when it is later finetuned on a more specific task such as caption generation.

The pretraining stage is computationally expensive. It is common practice to use existing pretrained models to bypass this stage and, thus, significantly reduce computational costs. In this chapter, we use a pretrained decoder-only Transformer such as GPT-2 [8] or Llama [4].

Similarly, the image encoder can also be derived from pretrained models. Instead of training an image encoder from scratch, we can leverage powerful pretrained vision models such as CLIP [9] or ViT [7].

2. Supervised finetuning

In this stage, we train both the image encoder and the text decoder on 400 million image–caption pairs. The image encoder improves its ability to encode image information effectively, and the text decoder learns to understand the sequence of image embeddings and generate a descriptive caption.

ML objective and loss function

The text decoder generates the caption one token at a time. Consistent with previous chapters, we use next-token prediction as our ML objective and employ cross-entropy loss [10] to guide the training process.

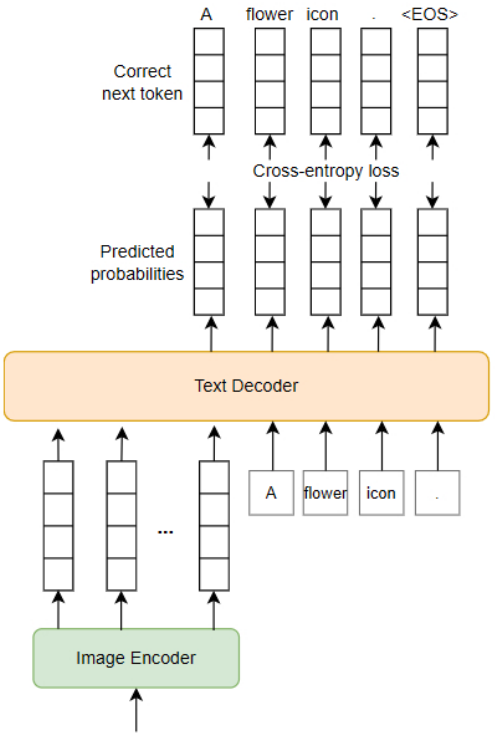


Figure 12: Loss calculation over the predicted probabilities

Sampling

During sampling, the caption tokens are generated one at a time.

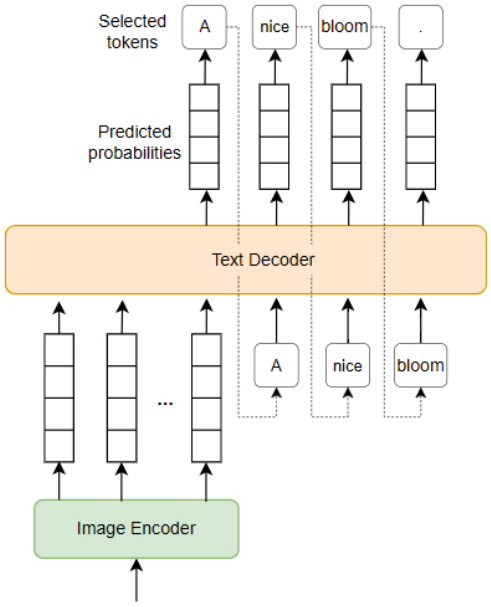


Figure 13: Generating a caption given an input image

While stochastic sampling methods can create creative captions, beam search ensures predictability. We use beam search for our image captioning system for the following reasons:

- **Quality:** Beam search typically generates higher-quality captions, which is critical for accurately describing image content.
- **Consistency:** The deterministic nature of beam search ensures that the model always produces the same caption for the same image. This consistency is crucial for image captioning.

- **Coherence:** Beam search typically produces coherent captions, which is important for image captioning. This avoids sudden topic changes or contradictions such as "A person is walking house" or "A dog is reading a person."

Evaluation

Offline evaluation metrics

During offline evaluation, we assess the performance of the trained model on a validation dataset. This is achieved by comparing generated captions with reference (i.e., correct) captions and measuring their similarity.

Before exploring common metrics, let's review validation data. Validation data contains examples not seen by the model during training. Each example includes an image and a set of reference captions. These captions are typically collected by having multiple human annotators describe each image.

In image captioning systems, it's common to have multiple reference captions for each image. This benefits both training and evaluation for the following reasons:

- **Robust training:** Different people describe the same image in different ways. Multiple references allow the model to learn different ways of describing an image. This leads to a more robust model that is capable of describing images more accurately.
- **Comprehensive evaluation:** Multiple captions provide a more thorough assessment of a model's performance. Comparing the generated caption to several correct reference captions leads to a fairer evaluation.

Image	Reference captions
	Blooming tulip with green leaves
	Close-up of a blooming tulip.
	Single tulip with leaves.

Figure 14: An example of validation data

The following metrics are commonly used in the offline evaluation of image captioning models:

- BLEU
- ROUGE
- METEOR
- CIDEr

The first three metrics on the list are explored extensively in Chapter 3. In this chapter, we focus on CIDEr, which has been designed specifically to evaluate image captioning models.

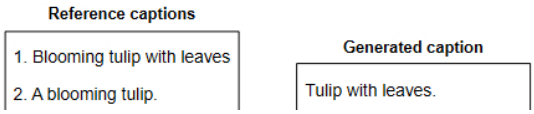
CIDEr

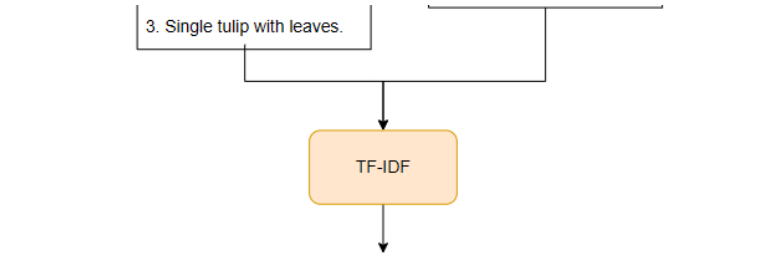
CIDEr [11] is a popular metric for evaluating image captioning models. It uses consensus to evaluate the similarity of a generated caption to a set of reference captions. CIDEr gives higher scores to captions that are similar to multiple reference captions rather than just one. For a single example, CIDEr is calculated in three steps:

1. Represent captions using **T**erm **F**requency–**I**nverse **D**ocument **F**requency (TF-IDF)
2. Calculate similarities
3. Aggregate the similarity scores

1. Represent captions using TF-IDF

In the first step, we convert the generated caption and each reference caption into numerical representations using TF-IDF. TF-IDF evaluates a word's importance to a document by considering how frequently it appears in that document and how common or rare it is across the entire corpus. These importance scores are used to represent a sentence numerically. To learn more about TF-IDF, refer to [12][13].





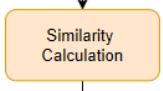
	a	blooming	leaves	single	tulip	leaves
Ref 1: “Blooming tulip with leaves”	0.00	0.52	0.43	0.00	0.36	0.43
Ref 2: “A blooming tulip”	0.70	0.50	0.00	0.00	0.50	0.00
Ref 3: “Single tulip with leaves”	0.00	0.00	0.57	0.57	0.34	0.57
Generated: “Tulip with leaves”	0.00	0.00	0.70	0.00	0.50	0.50

Figure 15: TF-IDF converting captions to numerical representations

2. Calculate similarity

Next, we calculate the similarity between the generated caption and each reference caption. We do this by computing the cosine similarity between their TF-IDF representations.

	a	blooming	leaves	single	tulip	leaves
Ref 1: “Blooming tulip with leaves”	0.00	0.52	0.43	0.00	0.36	0.43
Ref 2: “A blooming tulip”	0.70	0.50	0.00	0.00	0.50	0.00
Ref 3: “Single tulip with leaves”	0.00	0.00	0.57	0.57	0.34	0.57
Generated: “Tulip with leaves”	0.00	0.00	0.70	0.00	0.50	0.50



Pair	Cosine similarity score
<Reference 1, Generated>	0.688
<Reference 2, Generated>	0.257
<Reference 3, Generated>	0.766

Figure 16: Calculation of cosine similarity between generated and reference captions

A higher cosine similarity score (i.e., a score closer to 1) indicates greater similarity, while a lower value (closer to 0) indicates lesser similarity.

3. Aggregate the similarity scores

Once we have the cosine similarity scores between the generated caption and each of the reference captions, we take an average of these scores. This average score reflects the overall similarity between the generated caption and the reference captions.

$$CIDER_i = \frac{0.688 + 0.257 + 0.766}{3} = 0.570$$

The final CIDEr score is calculated by averaging the similarity scores for all generated captions in the validation dataset. This provides a single metric to evaluate the model's overall performance.

Let’s see some of the pros and cons of the CIDEr metric.

Pros:

- **Consensus-based:** CIDEr emphasizes consensus by rewarding captions that are similar to multiple reference captions. This leads to a more reliable evaluation of a model's performance.
- **Sensitive to important words**: TF-IDF assigns more weight to unique words in their representation. This ensures that the CIDEr score reflects the importance of words and rewards captions that use these words

that the CIDEr score reflects the importance of words and rewards captions that use those words.

- **Robust to different caption variations:** CIDEr is robust to different variations of generations since it is calculated based on multiple reference captions.

Cons:

- **Computationally complex:** Calculating TF-IDF representations in large datasets can be computationally expensive.
- **Sensitive to the quality of reference captions:** The quality and diversity of reference captions impact the CIDEr score. Poor references can lead to misleading evaluations.
- **Penalizes novel yet accurate captions:** CIDEr may penalize creative or novel phrases that are still accurate but are not present in the reference set.
- **Lack of semantic understanding:** CIDEr relies on TF-IDF to measure the similarity between two sentences. This might not always capture the semantic similarity when captions are textually similar but semantically different. For example, "Coffee on top of the table" and "Table on top of the coffee" might have similar TF-IDF representations due to similar words, but they are not semantically similar.

Online evaluation metrics

Online evaluation metrics are important for assessing the performance of ML systems. However, they are often not the primary focus in image captioning systems for two main reasons. First, image captioning systems are usually part of a bigger system, making it harder to collect user interaction data. Second, collecting feedback from users is challenging. Unlike tasks where we can easily measure user satisfaction, evaluating image caption quality requires subjective judgment, which, by definition, varies between users. For example, a caption might be acceptable to one user but not to another, depending on their personal interpretation of the image.

In summary, standard offline metrics remain the primary method for evaluating our image captioning system. For the few use cases where image captioning impacts user experience directly, engagement metrics and user feedback can provide valuable insights into the system's performance.

Overall ML System Design

Building an image captioning system is more than just training a model. It requires various components working together. In this section, we discuss the following key components essential for building an image captioning system:

- Image preprocessing
- Caption generator
- Post-processing

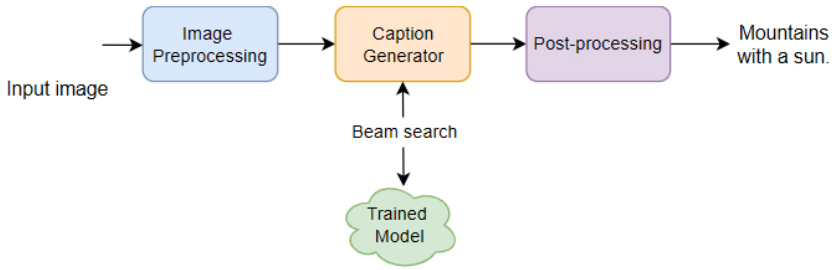


Figure 17: Image captioning overall design

Let's briefly explore each component and understand its role.

Image preprocessing

Image preprocessing is the initial step that prepares an input image for the trained model. This involves resizing images to a standard size, converting them into a consistent format, and standardizing pixel values. This step ensures that images are consistent with what the model expects as input.

Caption generator

The caption generator is the core component that produces captions based on the prepared image. This component interacts with the trained model and employs beam search to generate a coherent caption. If the cumulative probability

interacts with the trained model and employs beam search to generate a coherent caption. If the cumulative probability of the generated caption falls below a predefined confidence threshold, the name suggestion is disabled; otherwise, the caption is passed to the post-processing component. This ensures that the system avoids producing irrelevant captions for ambiguous images.

Post-processing

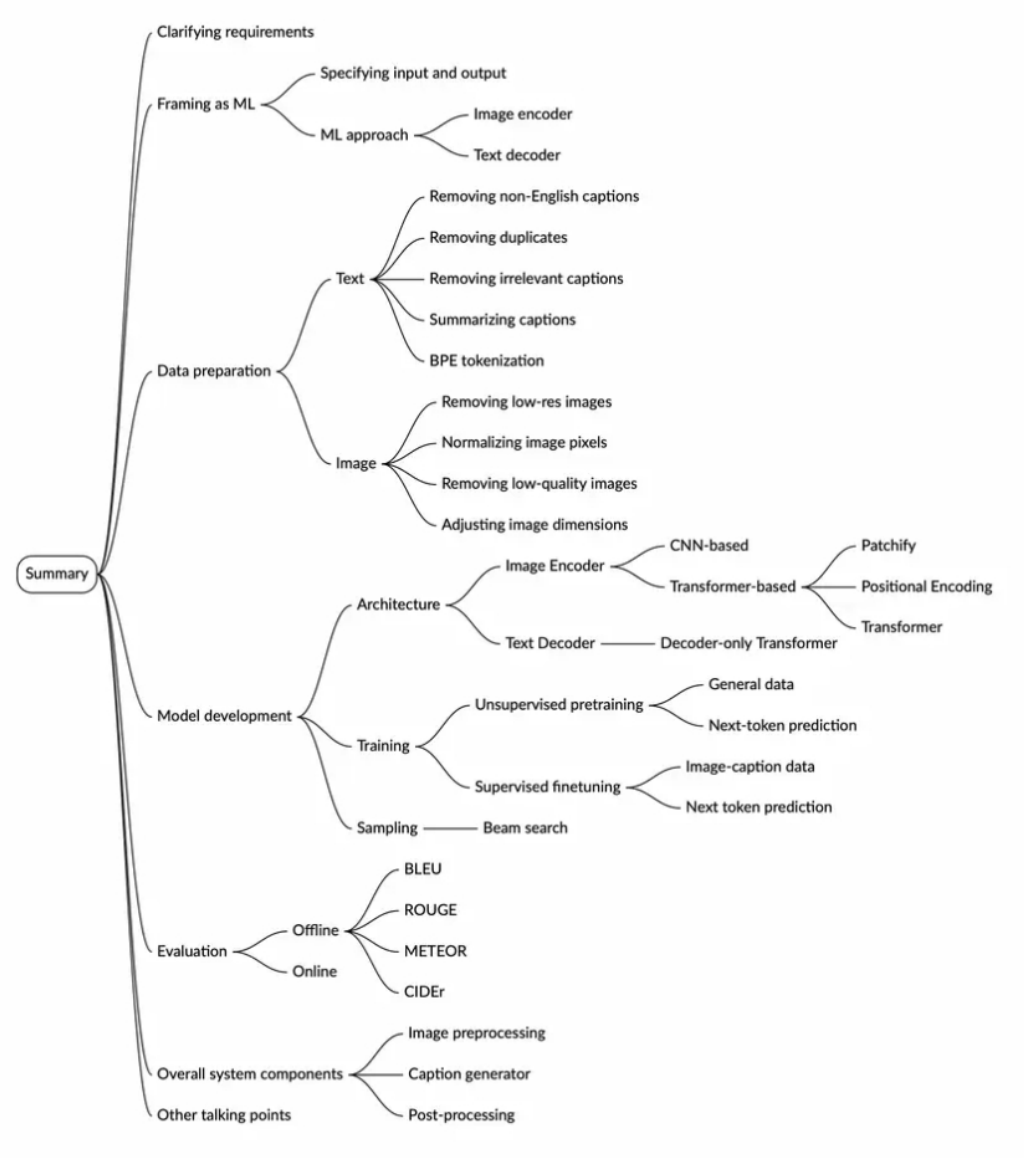
The post-processing component identifies biased terms or phrases in the caption and replaces them with neutral alternatives. This ensures fairness and inclusivity in generated captions. Additionally, it checks for the presence of offensive words and disables the name suggestion service if any are found.

Other Talking Points

If the interview finishes early, you might want to bring up the following topics:

- Extending the image captioner to support other tasks, such as visual question answering (VQA) [14].
- Adapting models to caption images from various domains [15].
- Generating captions in multiple languages using multilingual datasets and cross-lingual transfer learning [16].
- Optimization techniques for caption generation on edge devices [17].
- Generating and ranking multiple plausible captions based on relevance [18].
- Details of BLIP-2 and BLIP-3 methods and additional loss functions utilized for improving captioning [1] [2].

Summary



Reference Material

[1] BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. https://arxiv.org/abs/2301.12597.

[2] xGen-MM (BLIP-3): A Family of Open Large Multimodal Models. https://www.arxiv.org/abs/2408.08872.

[3] InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks. https://arxiv.org/abs/2312.14238.

[4] Meta's Llama. https://llama.meta.com/.

[5] Byte-pair encoding tokenization. https://huggingface.co/learn/nlp-course/en/chapter6/5.

[6] LAION-5B: An open large-scale dataset for training next generation image-text models. https://arxiv.org/abs/2210.08402.

[7] An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. https://arxiv.org/abs/2010.11929.

[8] Language Models are Unsupervised Multitask Learners. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[9] Learning Transferable Visual Models From Natural Language Supervision. https://arxiv.org/abs/2103.00020.

[10] Cross-entropy. https://en.wikipedia.org/wiki/Cross-entropy.

[11] CIDEr: Consensus-based Image Description Evaluation. https://arxiv.org/abs/1411.5726.

[12] TF-IDF introduction. https://web.stanford.edu/class/cs276/19handouts/lecture6-tfidf-1per.pdf.

[13] TF-IDF. https://en.wikipedia.org/wiki/Tf%E2%80%93idf.

[14] Visual question answering introduction. https://huggingface.co/tasks/visual-question-answering.

[15] Cross-Domain Image Captioning with Discriminative Finetuning. https://arxiv.org/abs/2304.01662.

[16] Crossmodal-3600 — Multilingual Reference Captions for Geographically Diverse Images. https://research.google/blog/crossmodal-3600-multilingual-reference-captions-for-geographically-diverse-images/.

[17] Efficient Image Captioning for Edge Devices. https://arxiv.org/abs/2212.08985.

[18] Ensemble model using an image captioning and ranking example. https://cloud.google.com/dataflow/docs/notebooks/run_inference_multi_model.

Mark as Complete